

Booth Multiplier Behavioural

Prompt Strategy: Zero Shot

Prompt:

Generate synthesizable Verilog code for a 16-bit signed Booth Multiplier.

Requirements:

- Implement Radix-2 Booth multiplication.
- Use Behavioral RTL style.
- Target FPGA: Xilinx Artix-7 xc7a100tcsg324-2.
- Use procedural logic where appropriate.
- Inputs:
 - input signed [15:0] multiplicand
 - input signed [15:0] multiplier
- Output:
 - output signed [31:0] product
- Generate synthesizable Verilog only.
- Output code only.

Code:

```
module booth_multiplier_16bit (  
    input signed [15:0] multiplicand,  
    input signed [15:0] multiplier,  
    output reg signed [31:0] product
```

);

integer i;

reg signed [31:0] A;

reg signed [16:0] Q;

reg signed [31:0] M;

always @* begin

A = 32'sd0;

M = {{16{multiplicand[15]}}, multiplicand};

Q = {multiplier, 1'b0};

for (i = 0; i < 16; i = i + 1) begin

case (Q[1:0])

2'b01: A = A + M;

2'b10: A = A - M;

default: A = A;

endcase

Q = {A[0], Q[16:1]};

A = A >>> 1;

end

product = {A[15:0], Q[16:1]};

end

endmodule

Prompt Strategy: Few Shot

Prompt:

Example of Behavioral RTL coding style:

```
module adder(  
    input [7:0] a,  
    input [7:0] b,  
    output logic [8:0] sum  
);  
  
always_comb begin  
    sum = a + b;  
end  
  
endmodule
```

Using the same Behavioral RTL style shown above, generate synthesizable Verilog code for a 16-bit signed Booth Multiplier.

Requirements:

- Radix-2 Booth multiplication.
- Behavioral RTL.
- Use procedural logic.

- Target FPGA: xc7a100tcsg324-2.
- Output code only.

Code:

```
module booth_multiplier_16bit (  
    input signed [15:0] multiplicand,  
    input signed [15:0] multiplier,  
    output logic signed [31:0] product  
);  
  
    integer i;  
  
    logic signed [32:0] A;  
    logic signed [32:0] M;  
    logic signed [32:0] Q;  
  
    always_comb begin  
        M = {multiplicand, 17'b0};  
        Q = {{16{1'b0}}, multiplier, 1'b0};  
  
        for (i = 0; i < 16; i = i + 1) begin  
            case (Q[1:0])  
                2'b01: Q = Q + M;  
                2'b10: Q = Q - M;  
                default: ;  
            end  
        end  
    end  
endmodule
```

```
        endcase

        Q = $signed(Q) >>> 1;

    end

    product = Q[32:1];

end

endmodule
```

Prompt Strategy: Chain of Thought

Prompt:

Design a 16-bit signed Booth Multiplier.

Before generating RTL:

1. Analyze the Booth multiplication algorithm.
2. Determine Booth encoding operations.
3. Determine the iteration process.
4. Determine arithmetic shift operations.
5. Determine final product generation.
6. Plan the behavioral implementation.

After completing the reasoning process, generate synthesizable Behavioral Verilog.

Requirements:

- Radix-2 Booth multiplication.
- Behavioral RTL.
- Use procedural constructs.
- Target FPGA: xc7a100tcs9324-2.
- Output complete code only.

Code:

```
module booth_multiplier_16bit (  
    input signed [15:0] multiplicand,  
    input signed [15:0] multiplier,  
    output reg signed [31:0] product  
);  
  
    integer i;  
  
    reg signed [16:0] A;  
    reg signed [16:0] M;  
    reg signed [15:0] Q;  
    reg          Q_1;  
  
    always @(*) begin  
        A = 17'sd0;  
        M = {multiplicand[15], multiplicand};
```

```

Q  = multiplier;

Q_1 = 1'b0;


for (i = 0; i < 16; i = i + 1) begin

    case ({Q[0], Q_1})

        2'b01: A = A + M;

        2'b10: A = A - M;

        default: A = A;

    endcase


    Q_1 = Q[0];

    Q  = {A[0], Q[15:1]};

    A  = {A[16], A[16:1]};

end


product = {A[15:0], Q};

end


endmodule

```

Prompt Strategy: Role Prompting

Prompt:

You are a Senior FPGA Design Engineer specializing in arithmetic units and DSP architectures for Xilinx FPGAs.

Generate production-quality synthesizable Verilog for a 16-bit signed Booth Multiplier.

Requirements:

- Radix-2 Booth multiplication.
- Behavioral RTL style.
- Use procedural implementation.
- Use arithmetic shift operations where appropriate.
- Optimized for Xilinx Artix-7 xc7a100tcsg324-2.
- Output code only.

Code:

```
module booth_multiplier_16bit (  
    input signed [15:0] multiplicand,  
    input signed [15:0] multiplier,  
    output reg signed [31:0] product  
);  
  
    integer i;  
  
    reg signed [16:0] m_ext;  
    reg signed [33:0] booth_reg;  
  
    always @* begin  
        m_ext = {multiplicand[15], multiplicand};  
        booth_reg = {17'sd0, multiplier, 1'b0};
```

```

for (i = 0; i < 16; i = i + 1) begin

    case (booth_reg[1:0])

        2'b01: booth_reg[33:17] = booth_reg[33:17] + m_ext;

        2'b10: booth_reg[33:17] = booth_reg[33:17] - m_ext;

        default: ;

    endcase

    booth_reg = $signed(booth_reg) >>> 1;

end

product = booth_reg[32:1];

end

endmodule

```

Prompt Strategy: Instruction Format

Prompt:

```

module booth_multiplier_16bit (
    input signed [15:0] multiplicand,
    input signed [15:0] multiplier,
    output reg signed [31:0] product
);

integer i;

```

```

reg signed [32:0] A;

reg signed [16:0] M;

reg signed [16:0] Q;


always @(*) begin

    A = 33'sd0;

    M = {multiplicand, 1'b0};

    Q = {multiplier, 1'b0};


    for (i = 0; i < 16; i = i + 1) begin

        case (Q[1:0])

            2'b01: A = A + {{16{M[16]}}, M};

            2'b10: A = A - {{16{M[16]}}, M};

            default: A = A;

        endcase


        {A, Q} = $signed({A, Q}) >>> 1;

    end


    product = {A[15:0], Q[16:1]};

end


endmodule

```

Code:

```

module booth_multiplier_16bit (

```

```
input signed [15:0] multiplicand,  
input signed [15:0] multiplier,  
output reg signed [31:0] product  
);
```

```
integer i;  
reg signed [32:0] A;  
reg signed [16:0] M;  
reg signed [16:0] Q;
```

```
always @(*) begin
```

```
    A = 33'sd0;
```

```
    M = {multiplicand, 1'b0};
```

```
    Q = {multiplier, 1'b0};
```

```
    for (i = 0; i < 16; i = i + 1) begin
```

```
        case (Q[1:0])
```

```
            2'b01: A = A + {{16{M[16]}}, M};
```

```
            2'b10: A = A - {{16{M[16]}}, M};
```

```
            default: A = A;
```

```
        endcase
```

```
        {A, Q} = $signed({A, Q}) >>> 1;
```

```
    end
```

```
    product = {A[15:0], Q[16:1]};
```

end

endmodule

Prompt Strategy: Negative Prompting

Prompt:

Generate synthesizable Verilog for a 16-bit signed Booth Multiplier.

Requirements:

- Radix-2 Booth multiplication.
- Behavioral RTL.

Do NOT:

- Use the multiplication operator (*).
- Use assign-based full dataflow implementations.
- Use structural gate-level descriptions.
- Use unnecessary module hierarchies.
- Use non-synthesizable constructs.

Must:

- Use procedural logic.
- Implement Booth encoding.
- Implement arithmetic shift operations.
- Implement iterative Booth algorithm behavior.

Target FPGA:

xc7a100tcsg324-2

Output code only.

Code:

```
module booth_multiplier_16bit (  
    input signed [15:0] multiplicand,  
    input signed [15:0] multiplier,  
    output reg signed [31:0] product  
);  
  
    integer i;  
  
    reg signed [32:0] booth_reg;  
    reg signed [16:0] m_ext;  
  
    always @(*) begin  
        m_ext = {multiplicand[15], multiplicand};  
        booth_reg = {17'd0, multiplier, 1'b0};  
  
        for (i = 0; i < 16; i = i + 1) begin  
            case (booth_reg[1:0])  
                2'b01: booth_reg[32:16] = booth_reg[32:16] + m_ext;  
                2'b10: booth_reg[32:16] = booth_reg[32:16] - m_ext;
```

```
        default: ;

    endcase

    booth_reg = booth_reg >>> 1;

end

    product = booth_reg[32:1];

end

endmodule
```

Prompt Strategy: Constraints First

Prompt:

Design Constraints:

1. Behavioral RTL only.
2. Synthesizable on Xilinx Artix-7 xc7a100tcsg324-2.
3. Radix-2 Booth algorithm required.
4. Signed 16-bit multiplication.
5. Output width must be 32 bits.
6. Procedural implementation required.
7. Arithmetic shift operations required.
8. Booth encoding required.
9. No multiplication operator (*).
10. Synthesizable Verilog only.

Task:

Generate a Booth Multiplier satisfying all constraints above.

Output Verilog code only.

Code:

```
module booth_multiplier_16bit (  
    input signed [15:0] multiplicand,  
    input signed [15:0] multiplier,  
    output reg signed [31:0] product  
);
```

```
    integer i;
```

```
    reg signed [15:0] A;
```

```
    reg signed [15:0] Q;
```

```
    reg signed [15:0] M;
```

```
    reg          Q_1;
```

```
    reg signed [32:0] booth_reg;
```

```
    always @(*) begin
```

```
        A = 16'sd0;
```

```

Q  = multiplier;
M  = multiplicand;
Q_1 = 1'b0;

for (i = 0; i < 16; i = i + 1) begin
    case ({Q[0], Q_1})
        2'b01: A = A + M;
        2'b10: A = A - M;
        default: A = A;
    endcase

    booth_reg = {A, Q, Q_1};
    booth_reg = booth_reg >>> 1;

    A  = booth_reg[32:17];
    Q  = booth_reg[16:1];
    Q_1 = booth_reg[0];
end

product = {A, Q};
end

endmodule

```

Prompt Strategy: Self Planning

Prompt:

Generate a 16-bit signed Booth Multiplier.

First create an implementation plan.

The plan should identify:

- Booth encoding process
- Iterative multiplication process
- Arithmetic shift strategy
- Register organization
- Product generation method

After creating the plan, implement the design according to the plan.

Requirements:

- Radix-2 Booth multiplication.
- Behavioral RTL.
- Synthesizable Verilog.
- No multiplication operator (*).
- Target FPGA: xc7a100tcsg324-2.

Output complete code.

Code:

```
module booth_multiplier_16bit (
```

```
input signed [15:0] multiplicand,  
input signed [15:0] multiplier,  
output reg signed [31:0] product  
);
```

```
integer i;
```

```
reg signed [16:0] M;
```

```
reg signed [16:0] A;
```

```
reg signed [15:0] Q;
```

```
reg          Q_1;
```

```
reg signed [33:0] booth_reg;
```

```
always @(*) begin
```

```
    M = {multiplicand[15], multiplicand};
```

```
    A = 17'sd0;
```

```
    Q = multiplier;
```

```
    Q_1 = 1'b0;
```

```
    for (i = 0; i < 16; i = i + 1) begin
```

```
        case ({Q[0], Q_1})
```

```
            2'b01: A = A + M;
```

```
            2'b10: A = A - M;
```

```
            default: A = A;
```

```

        endcase

        booth_reg = {A, Q, Q_1};

        booth_reg = booth_reg >>> 1;

        A  = booth_reg[33:17];
        Q  = booth_reg[16:1];
        Q_1 = booth_reg[0];
    end

    product = {A[15:0], Q};
end

endmodule

```

Prompt Strategy: Iterative Correction

Prompt:

Generate synthesizable Verilog for a 16-bit signed Booth Multiplier.

After generating the initial design:

1. Review Booth algorithm correctness.
2. Verify signed multiplication support.
3. Verify arithmetic shift behavior.

4. Verify synthesizability.
5. Verify correct output width.
6. Verify procedural implementation.
7. Correct any identified issues.

Then provide the final corrected version.

Requirements:

- Radix-2 Booth multiplication.
- Behavioral RTL.
- No multiplication operator (*).
- Target FPGA: xc7a100tcsg324-2.
- Output final code only.

Code:

```
module booth_multiplier_16 (  
    input signed [15:0] multiplicand,  
    input signed [15:0] multiplier,  
    output reg signed [31:0] product  
);  
  
    integer i;  
  
    reg signed [16:0] M;  
    reg signed [16:0] A;
```

```

reg signed [15:0] Q;

reg          Q_1;

always @(*) begin

    M  = {multiplicand[15], multiplicand};

    A  = 17'sd0;

    Q  = multiplier;

    Q_1 = 1'b0;

    for (i = 0; i < 16; i = i + 1) begin

        case ({Q[0], Q_1})

            2'b01: A = A + M;

            2'b10: A = A - M;

            default: A = A;

        endcase

        {A, Q, Q_1} = $signed({A, Q, Q_1}) >>> 1;

    end

    product = {A[15:0], Q};

end

endmodule

```

Prompt Strategy: Hybrid

Prompt:

You are a Senior FPGA Design Engineer specializing in FPGA arithmetic accelerators.

TASK:

Generate synthesizable Verilog for a 16-bit signed Booth Multiplier.

Before coding:

1. Analyze the Booth multiplication algorithm.
2. Create an implementation plan.
3. Determine Booth encoding strategy.
4. Determine arithmetic shift strategy.
5. Determine register organization.
6. Determine final product generation.

CONSTRAINTS:

- Behavioral RTL only.
- Radix-2 Booth multiplication.
- Signed 16-bit operands.
- 32-bit product output.
- No multiplication operator (*).
- Synthesizable Verilog.
- Target FPGA: Xilinx Artix-7 xc7a100tcsg324-2.

SELF-CHECK:

- Verify Booth algorithm correctness.
- Verify signed arithmetic.
- Verify arithmetic shift operations.
- Verify synthesizability.
- Verify correct output width.

OUTPUT:

Verilog code only.

Code:

```
module booth_multiplier_16bit (
    input signed [15:0] multiplicand,
    input signed [15:0] multiplier,
    output reg signed [31:0] product
);

    integer i;

    reg signed [32:0] booth_reg;
    reg signed [16:0] multiplicand_ext;

    always @* begin
        multiplicand_ext = {multiplicand[15], multiplicand};
```

```

// {A[16:0], Q[15:0], Q-1}

booth_reg = {17'd0, multiplier, 1'b0};

for (i = 0; i < 16; i = i + 1) begin
    case (booth_reg[1:0])
        2'b01: booth_reg[32:16] = booth_reg[32:16] + multiplicand_ext;
        2'b10: booth_reg[32:16] = booth_reg[32:16] - multiplicand_ext;
        default: ;
    endcase

    // Arithmetic right shift of {A,Q,Q-1}
    booth_reg = $signed(booth_reg) >>> 1;
end

// Final product = {A,Q}
product = booth_reg[32:1];
end

endmodule

```